Week 4 Worked Companion

Using the worked companion

Attempt the student exercises independently first. This companion contains hints and complete solutions. Read only as far as needed, then return to your attempt.

Examples are original teaching examples. Follow each official link for the source problem and its complete contract.

Student exercises

Instructor guide

Download all code

104 · Staged hints

LeetCode 104: Maximum Depth of Binary Tree · Easy

H1. What should the empty-tree call return?

H2. May an elif skip the right subtree after visiting the left?

H3. Derive depth(None)=0 and depth(node)=1+max(left,right) on small trees; the robust implementation uses an explicit (node,depth) stack.

104 · Correctness and cost

Structural induction proves the recurrence. The explicit stack carries the true root depth to every node and records its maximum.

n is node count, h height. O(n) time, O(h) DFS space, O(n) worst case.

104 · Counterexamples and checks

Python recursion depth may be below a 10^4-node chain; distinguish the small-tree recursive demonstration from the full-contract iterative version.

`root=[4,2,7,None,3] → 3`

Trace before execution: write down each state and the return value, then compare with the code. Follow-up: what changes if mutation is forbidden, a different output is requested, or structure costs change?

104 · Implementation

The implementation below is included in the code bundle.

```
def max_depth(root):
    """Explicit stack supports skew trees within the official size limit."""
    if root is None:
        return 0
    stack, best = [(root, 1)], 0
    while stack:
        node, depth = stack.pop()
        best = max(best, depth)
        if node.left:
            stack.append((node.left, depth + 1))
        if node.right:
            stack.append((node.right, depth + 1))
    return best
```

102 · Staged hints

LeetCode 102: Binary Tree Level Order Traversal · Medium

H1. What is the queue length before processing one level?

H2. What happens if the loop follows the growing queue length?

H3. Pop exactly the initial len(queue) nodes, appending children for the next level.

102 · Correctness and cost

At each outer-loop entry the queue contains one level. Processing its initial size leaves only the next level; left-first insertion preserves order.

n is node count, w maximum width. O(n) time, O(w) queue space, O(n) output.

102 · Counterexamples and checks

list.pop(0) shifts the remaining entries; repeated use can reach $O(n^2)$ on wide trees.

```
root=[4,2,7,None,3] → [[4],[2,7],[3]]
```

Trace before execution: write down each state and the return value, then compare with the code. Follow-up: what changes if mutation is forbidden, a different output is requested, or structure costs change?

102 · Implementation

The implementation below is included in the code bundle.

```
def level_order(root):
    if root is None:
        return []
    queue, result = deque([root]), []
    while queue:
        level = []
        for _ in range(len(queue)):
            node = queue.popleft()
            level.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        result.append(level)
    return result
```

98 · Staged hints

LeetCode 98: Validate Binary Search Tree · Medium

H1. Besides parent 10, which ancestor constrains 6?

H2. How must a child failure propagate?

H3. Carry an open interval (lower,upper); tighten the upper bound on the left and lower bound on the right.

98 · Correctness and cost

The interval is the intersection of all ancestor constraints. Tightening it preserves that meaning; any violation invalidates the whole tree.

n is node count, h height. O(n) time, O(h) explicit DFS space, O(n) worst case.

98 · Counterexamples and checks

The example’s 6 is below its parent 10 but lies right of root 8; parent-only checks miss it.

`root=[8,3,10,None,None,6,12] → False`

Trace before execution: write down each state and the return value, then compare with the code. Follow-up: what changes if mutation is forbidden, a different output is requested, or structure costs change?

98 · Implementation

The implementation below is included in the code bundle.

```
def valid_bst(root):
    stack = [(root, float("-inf"), float("inf"))]
    while stack:
        node, lower, upper = stack.pop()
        if node is None:
            continue
        if not lower < node.val < upper:
            return False
        stack.append((node.right, node.val, upper))
        stack.append((node.left, lower, node.val))
    return True
```

230 · Staged hints

LeetCode 230: Kth Smallest Element in a BST · Medium

H1. What value order does left-node-right produce?

H2. Is k a visit count or a zero-based index?

H3. Use an explicit inorder stack; decrement k when visiting a node.

230 · Correctness and cost

Every left value is smaller and every right value larger. Induction makes inorder sorted, so the kth visit gives the answer.

n is node count, h height. O(h+k) time, O(n) worst case, O(h) space.

230 · Counterexamples and checks

If materializing inorder, index at k-1. That alternative uses O(n) auxiliary space and time.

```
root=[5,2,8,1,3], k=3 → 3
```

Trace before execution: write down each state and the return value, then compare with the code. Follow-up: what changes if mutation is forbidden, a different output is requested, or structure costs change?

230 · Implementation

The implementation below is included in the code bundle.

```
def kth_smallest(root, k):
    stack, node = [], root
    while stack or node:
        while node:
            stack.append(node)
            node = node.left
        node = stack.pop()
        k -= 1
        if k == 0:
            return node.val
        node = node.right
    raise ValueError("k exceeds tree size")
```

Execution and sources

```
cd code
python demo_07.py
python demo_08.py
python verify.py
```

Common objects and imports are in structures.py and solutions.py. On LeetCode, adapt the function to the official method signature.

Erik Demaine and Srini Devadas, Introduction to Algorithms, MIT 6.006 (Fall 2011).

Recursive depth derivation

A missing child contributes height 0. Every path starting at a nonempty node consists of that node followed by a path in one child, so its height is 1 plus the larger child height. Induction on subtree size proves the result: both child subtrees are smaller, and the base case is empty.

For `[4,2,7,None,3]`, nodes 3 and 7 return 1; node 2 returns 2; node 4 returns 3. The recursive demonstration uses O(h) call-stack space. The iterative solution stores pending nodes explicitly and works without increasing Python’s recursion limit.

Midpoint checks: answers and reasoning

1. `[4,2,7,None,3]`: depth 3, levels `[[4],[2,7],[3]]`. The path 4,2,3 contains three nodes.
2. `[8,3,10,None,None,6,12]`: invalid. The value 6 is in root 8's right subtree and must exceed 8, even though it is smaller than parent 10.
3. BST `[5,2,8,1,3]`: k=1,3,5 gives 1,3,8. Inorder visits `1,2,3,5,8`.

An empty tree returns depth 0 and levels `[]`. A single node has depth 1. Replacing a strict bound with an inclusive bound incorrectly accepts duplicate BST keys.

Reading the runnable code

`structures.py` defines `TreeNode` and `tree(values)`. The adapter consumes level-order values, using `None` for absent children. It allocates the input nodes; that allocation is separate from the algorithms' auxiliary-space bounds.

`solutions.py` imports `deque`. Its four iterative solutions preserve the input nodes and links. `depth_recursive` is a small-tree demonstration. For LeetCode, wrap the functions as `maxDepth`, `levelOrder`, `isValidBST`, and `kthSmallest` methods in `class Solution`, using the supplied `TreeNode`.

`verify.py --student` runs the same checks against your implementations. Local differential tests support debugging; they do not replace the correctness arguments above.